Hertie School

Deep Learning

Session 2: Deep neural networks (1)

September 10, 2024

Meet the Centre for Sustainability

Hertie School
Centre for Sustainability

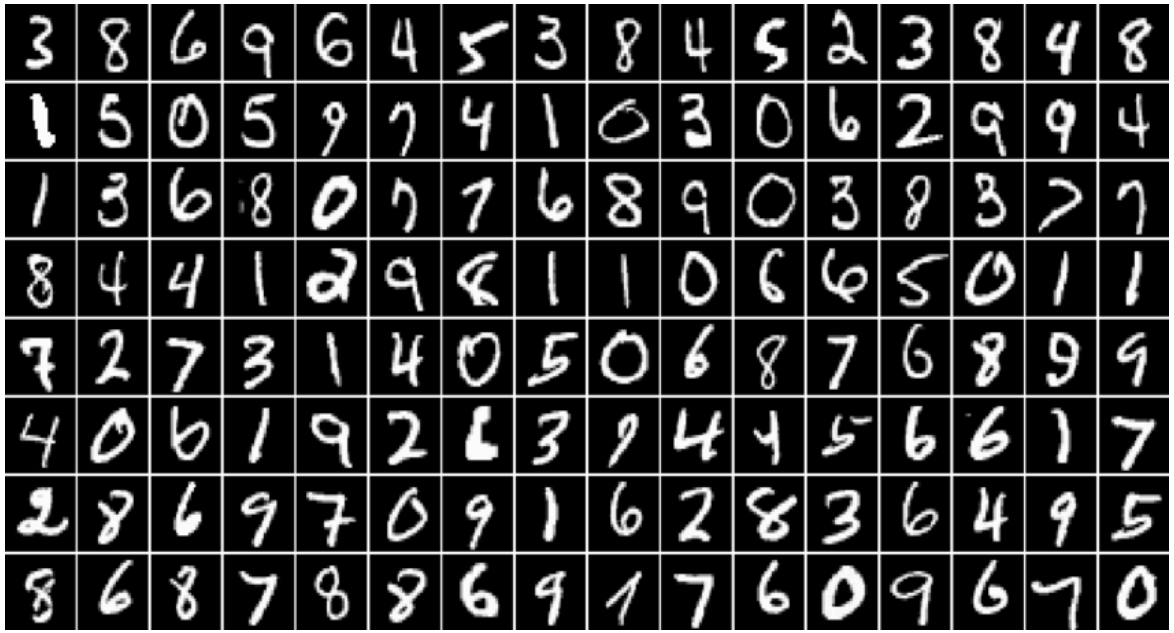
Wednesday September 18 | 3-4pm CET
Hertie School Forum

Logistics

Hertie School

Problem set 1 out soon

- Theoretical problems
- Neural network implementation from scratch for multi-class classification on MNIST



Syllabus

Hertie School

Deep learning theory

1. Deep learning in public policy
2. **Deep neural networks (1)**
3. Deep neural networks (2)

Applications

4. Computer vision (1): Convolutional neural networks
5. Computer vision (2): Modern CNNs, implementation, and applications
6. Sequence methods and time-series analysis
7. Natural language processing (1)
8. Natural language processing (2): Transformer models, implementation, and applications

- Midterm exam -

9. Further topics in deep learning

Deep learning and public policy

10. Policy approaches to regulate deep learning and responsible implementation in government
11. Deep learning in practice
12. Tutorial presentations

Learning objectives

Hertie School

Deep neural networks (1)

- Overview of how neural networks "learn"
- Structure of single-layer neural networks
- Gradient descent
- Training the network and back-propagation

Hertie School

High-level overview: How a neural network learns

Prediction with neural networks

Data point (x_i)

Model

black box

Neural network

Class label (y_i)

Dog

Task: Classification of images into “dog” or “cat”.

Training neural networks

Data point (x_i)

Model

Neural network

Predicted label ($\hat{y}_i$)

```
43% Dog
57% Cat
```

1) Feed-forward

takes an example, feeds it into a NN, predicts a vector that gives % of class labels
this assumes some preset config of the model, then does an initial attempt at a classification

Training neural networks

Hertie School

Data point (x_i)

Model

Neural network

Predicted label ($\hat{y}_i$)

43% Dog
57% Cat

Loss ($\hat{y}_i$, y_i)

Updating weights through back-propagation and gradient descent

2) Computing loss and back-propagation

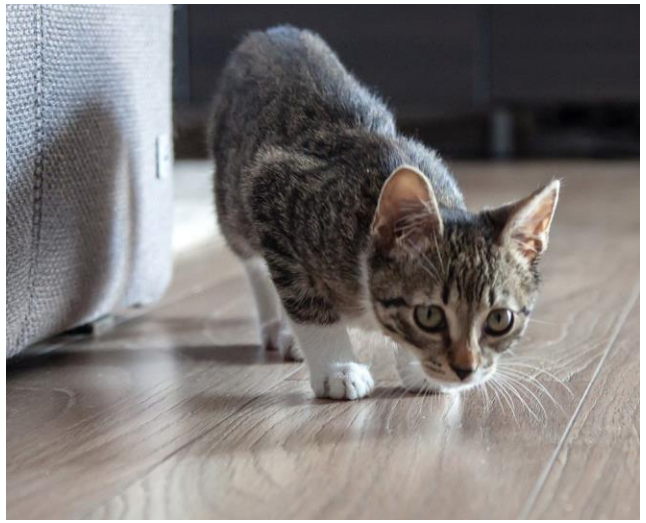
now we use this info to change our model

1) making a prediction, 2) see how wrong we are, 3) use that info to move in the right direction to be more right

Training neural networks

Hertie School

Data point (x_i)



Model

Neural network

Predicted label ($\hat{y}_i$)

39 % Dog
61 % Cat

Loss ($\hat{y}_i, y_i$)

Updating weights through back-propagation and gradient descent

Feed-forward and back-propagation with many more data points from training set

Hertie School

Development and testing of neural networks

Unseen data point (x_i)
a new image from test data

Model (stays fixed)

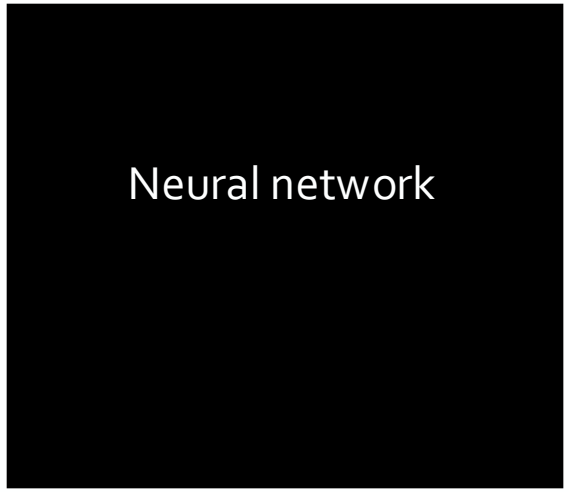
Neural network

Predicted label ($\hat{y}_i$)

35 % Dog
65 % Cat

- Hyperparameter tuning on validation set
- Reporting performance on test set

Unpacking a neural net



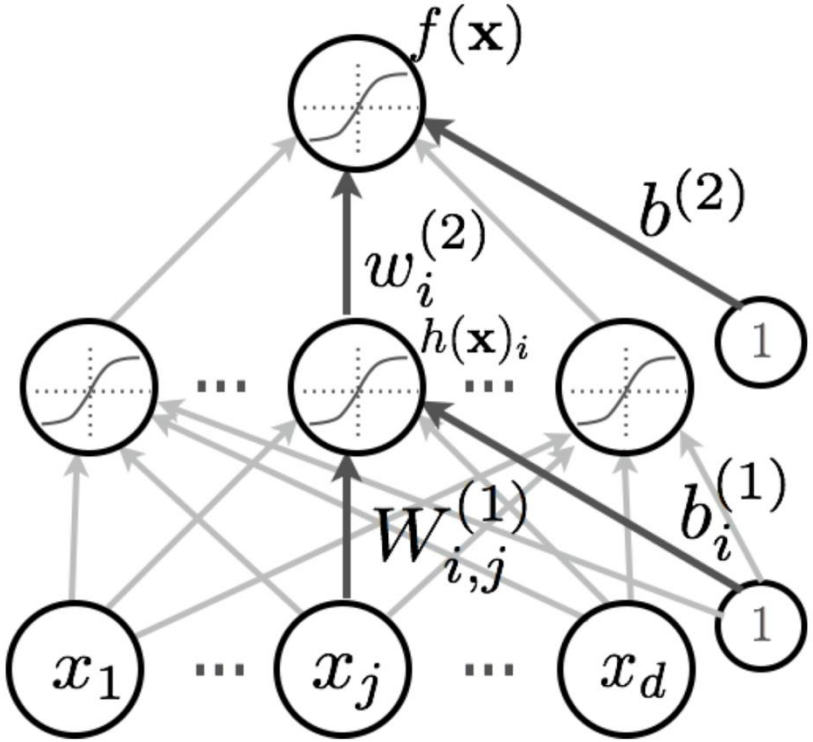


Image: Slides of R. Salakhutdinov

Hertie School

Single-layer neural networks

Adapted from Mathematics for Data Science GRAD-C23 (Spring 2022)

Based on Zhang et al. and other sources

Neuron

Inspired by how the brain works

- Information arrives from multiple sources in the dendrites
- Synaptic weights decide how much of this information it is actually used
- The information from multiple sources is then aggregated in the nucleus.
- Sent to axon terminals via the axon
- From axon terminal sent to muscle or other neuron
- Inspiration for modern deep learning comes from mathematics, linguistics, psychology, statistics, computer science, and many other fields.

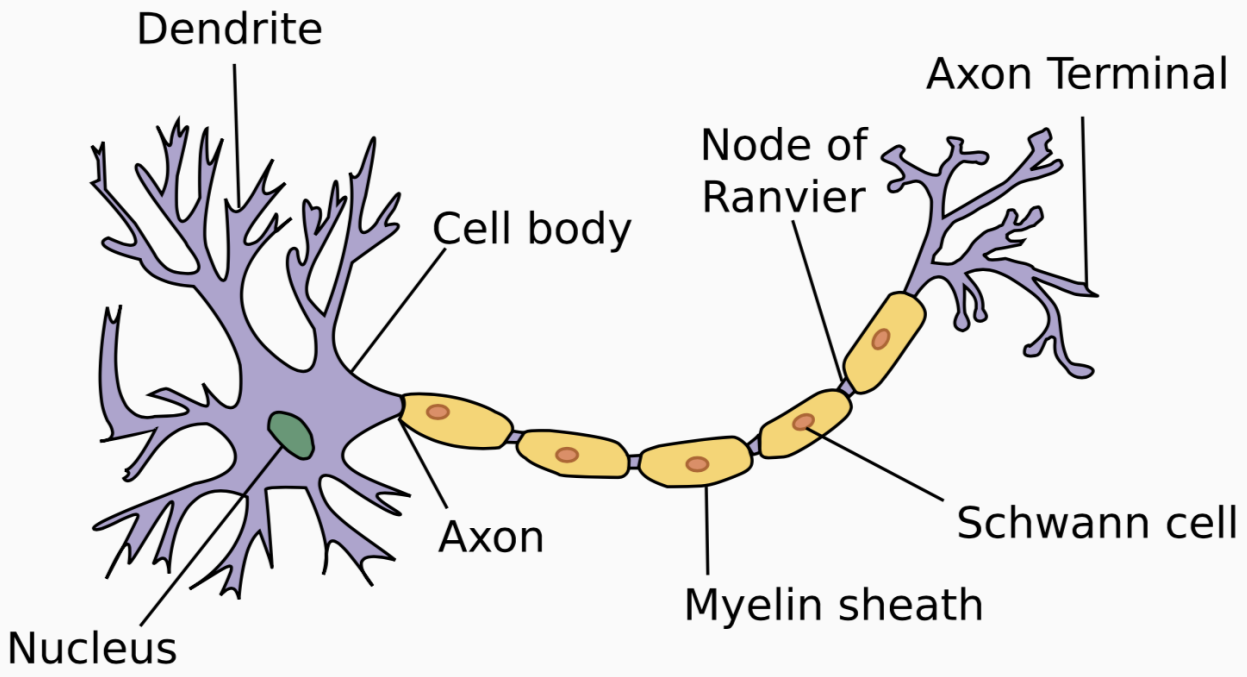


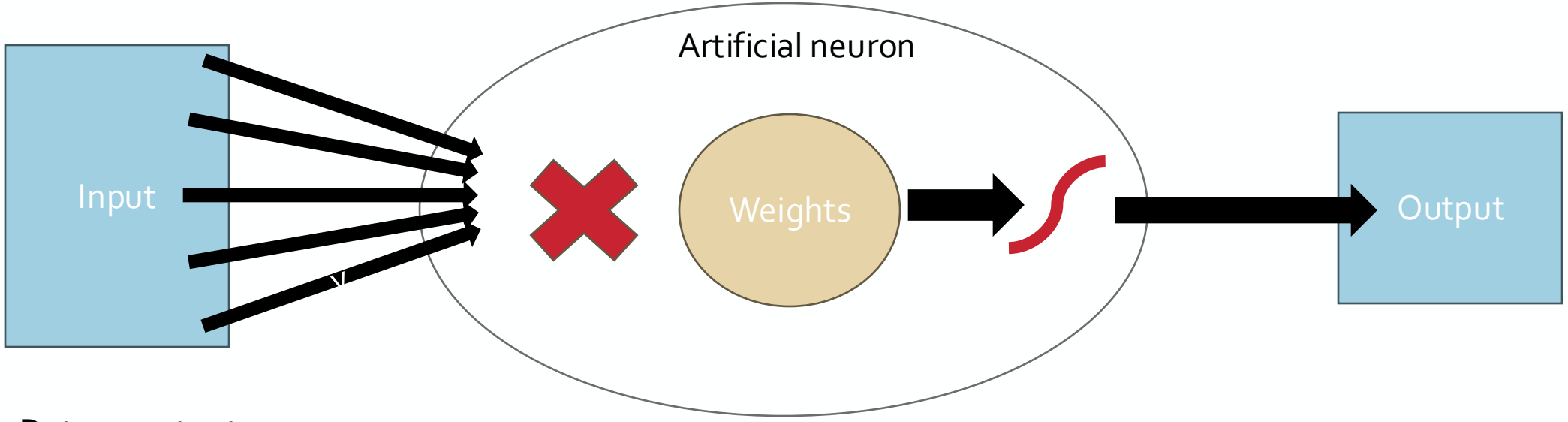
Fig. 3.1.3 The real neuron.

aggregates info then sends it on
Modern DL is an engineering discipline that has little to do with neuroscience

Image: Zhang et al.

Artificial neuron

Artificial neuron / (hidden) unit



Data or output from the previous step

info comes in from all the different features we have (if initial layer), or a signal from prev layer

Multiplication

multiply inputs w/ weights then apply function to results then give as output

A matrix or tensor of numbers

Nonlinear function

activation function

Prediction or signal to the next step

Artificial neuron

Beware that the number of input features are called differently across sources: $d, p, n, \ldots$

Artificial neuron / (hidden) unit

Neuron input activation for data $X \in \mathbb{R}^{n \times d}$:

$$a(X) = b + \sum_{j}^{d} w_j x_j = b + W^T X.$$

b = a bias term
just a constant we add

Neuron output activation:

$$h(X) = g\big(a(X)\big) = g\left(b + \sum_{j}^{d} w_j x_j\right).$$

where

- W^T are the weights (a vector of length d = dimension of inputs (# features))
- b is the bias term (scalar)
- $g(\cdot)$ is called the activation function

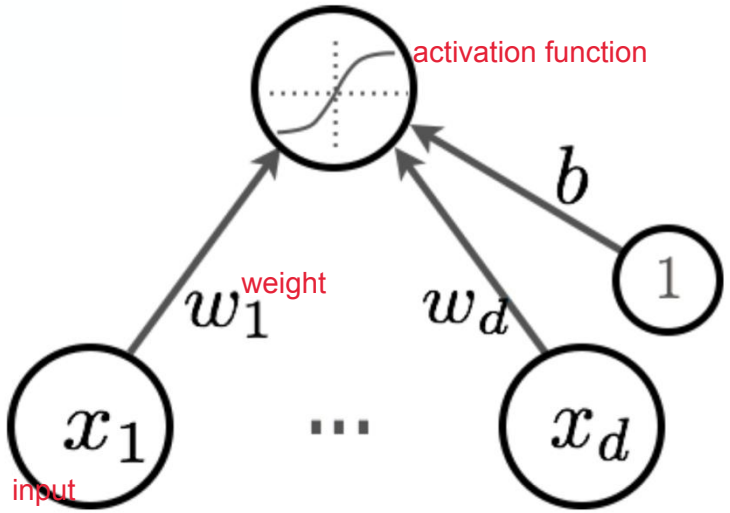


Image: Slides of R. Salakhutdinov

Single-layer neural network

Feed-forward neural network

We have input data X with d features. The neural network model ties many neurons (hidden units) together that each are a different transformation of the original features.

Hidden unit i:

$$h_i(X) = g\big(a_i(X)\big) = g\left(b_i^{[1]} + \sum_j^d w_{ij}^{[1]} x_j\right).$$

The **activation functions** $g(\cdot)$ in every hidden unit h_i can make it non-linear. The H **hidden units** feed into the output layer, which again consists of a linear combination of the hidden units, with an activation function $o(\cdot)$ applied to it:

$$f(X) = o\left(b^{[2]} + \sum_i^H w_i^{[2]} h_i(X)\right)$$

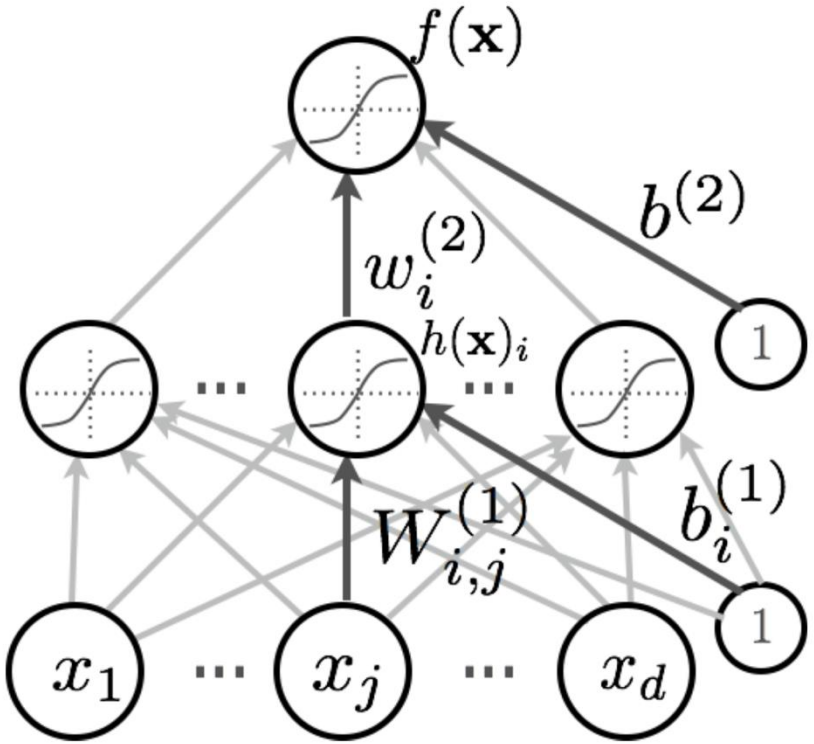


Image: Slides of R. Salakhutdinov

non linear activation function allows the NN to be expressive;
without the activation function they collapse and it becomes one big linear regression
w/o g we could collapse everything into a linear regression equation

Partner discussion

Hertie School

Which dimension does each of the weight vectors have?

We have input data X with d features. The neural network model ties many neurons (hidden units) together that each are a different transformation of the original features.

Hidden unit i:

$$h_i(X) = g\big(a_i(X)\big) = g\left(b_i^{[1]} + \sum_j^d w_{ij}^{[1]} x_j\right).$$

The **activation functions** $g(\cdot)$ in every hidden unit h_i can make it non-linear. The H **hidden units** feed into the output layer, which again consists of a linear combination of the hidden units, with an activation function $o(\cdot)$ applied to it:

$$f(X) = o\left(b^{[2]} + \sum_i^H w_i^{[2]} h_i(X)\right)$$

$$b^{[1]} \in \mathbb{R}^?, W^{[1]} \in \mathbb{R}^?$$
$$b^{[2]} \in \mathbb{R}^?, W^{[2]} \in \mathbb{R}^?$$

get the answers from the slides

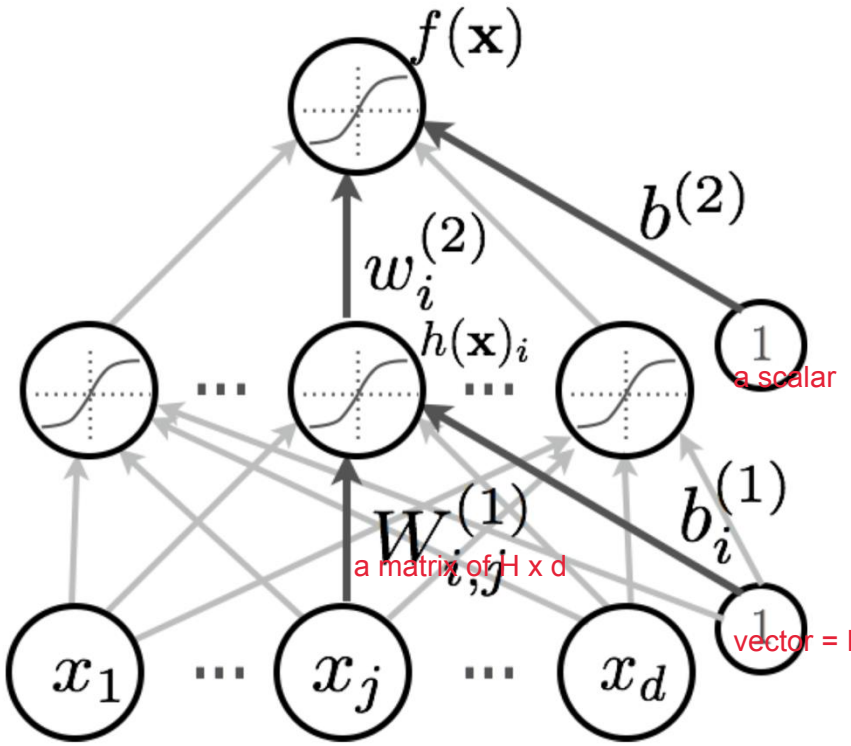


Image: Slides of R. Salakhutdinov

W[2] depends on the dimensions of the input and output layers

Hertie School

Single-layer neural network

Estimation from data

Single-layer neural network:

$$f(X) = o\left(a^{[2]}\right) = o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} h_i(X)\right) = o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} g\left(b_i^{[1]} + \sum_{j}^{d} w_{ij}^{[1]} x_j\right)\right)$$

- In theory, a single hidden layer with a large number of units has the ability to approximate most functions.
- We learn the **weights** from the data. b & Ws are learnt, the other parameters aren't
- All other parameters are set as **hyperparameters**.
- To learn the weights, we minimize a **loss function** using gradient descent
- When we apply this to neural networks and compute the gradient of the loss function, we call this **backpropagation** = how we turn info from the loss into updates of Ws and bs

Hertie School

Activation function

Motivation

Single layer neural network:

$$f(X) = o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} h_i(X)\right) = o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} g\left(b_i^{[1]} + \sum_{j}^{d} w_{ij}^{[1]} x_j\right)\right)$$

- A neural network needs **non-linear activation functions**, otherwise it is a linear model
- Activation functions can capture complex nonlinearities and interaction effects
- Think "artificial neural network": values of the activations $h_i(X) = g(a_i(X))$ in each "neuron" close to 1 are "firing", while those close to zero are silent
- We need a function that is low below a threshold and then high
- Popular activation functions:
    - Sigmoid
    - ReLU
    - Heaviside function
    - tanh

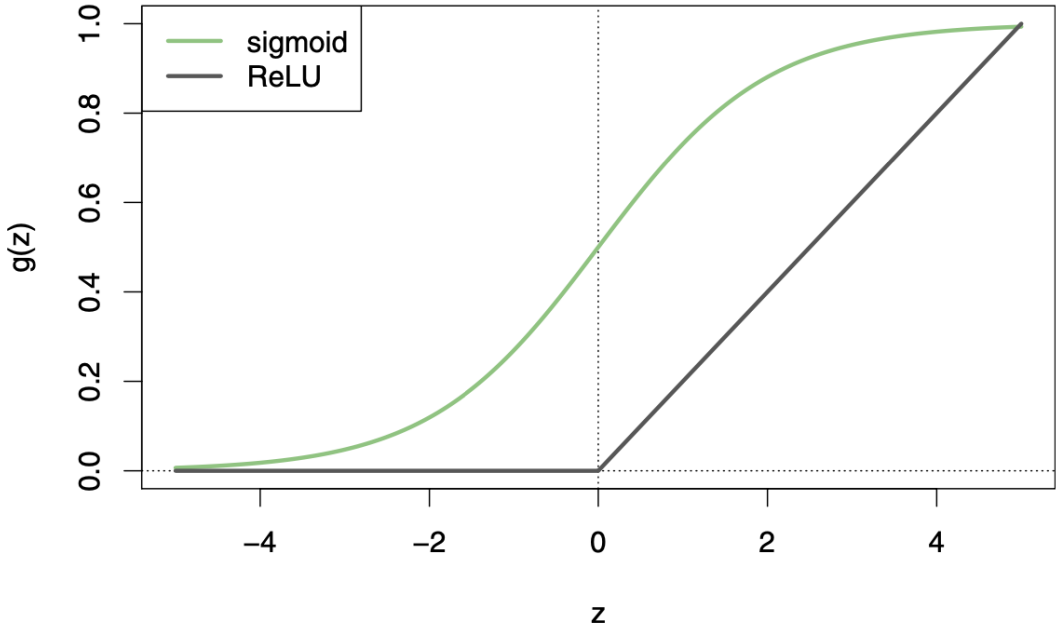
allow NN to display any complex function; in a lineae reg we have to add the interactions by hand, but the NN is so expressive that it can do itself

Activation function

Sigmoid

$$g(z) = \sigma(z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$$

- Also called logistic activation function
- Compare logistic function $f(x) = \frac{L}{1+e^{-k(x-x_0)}}$
- S-shaped curve
- Traditionally widely used



Comparing sigmoid and ReLU: In this plot, ReLU is scaled by a factor of 5 (above), original ReLU (below)

ReLU

$$g(z) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{otherwise} \end{cases}$$

it's either 0, and then
it's the identity

- ReLU = rectified linear unit
- ReLU activation can be computed and stored more efficiently than a sigmoid activation computationally more efficien + behaves better than sigmoid
- Widely used in deep learning

non linear becuase there's a break; it's not even differentiable

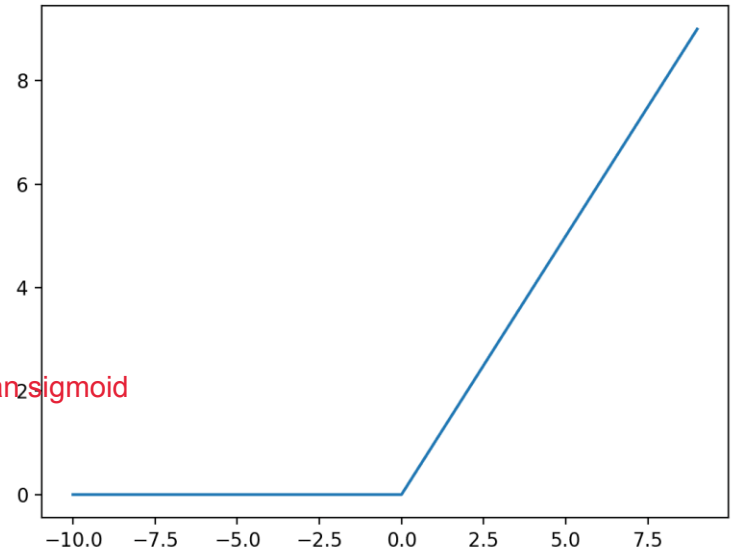


Image: ISLR

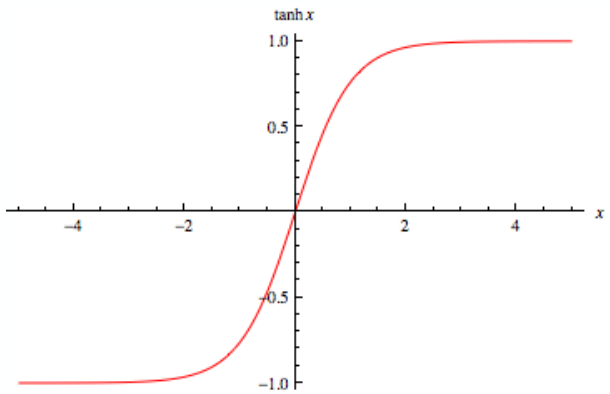
Activation function

Other activation functions not used as much

Hyperbolic tangent (tanh):

$$g(z) = \tanh z = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

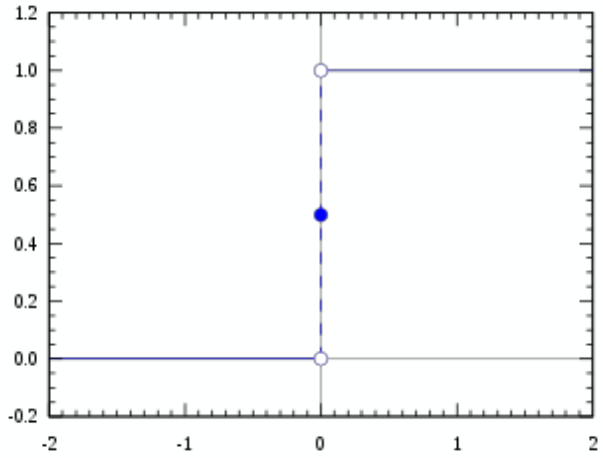
tanh:



Heaviside step function:

$$g(z) = H(z) = \mathrm{I}(z > 0)$$
$$= \begin{cases} 1 & \text{for } z > 0 \\ 0 & \text{for } z \leq 0 \end{cases}$$

Heaviside:



Linear:

$$g(z) = z$$

Typically, only used for output layers. this is just the identity; it's used for output layers

Output layer activation function

the activation transforms / scales the output of the neuron

Single output

Regression: We just use the identity $o\left(a^{[2]}\right) = a^{[2]}$

Binary classification: We can use the sigmoid $o\left(a^{[2]}\right) = \sigma(a^{[2]})$

Multiclass classification

Network with $\boldsymbol{K}$ **classes** for classification:

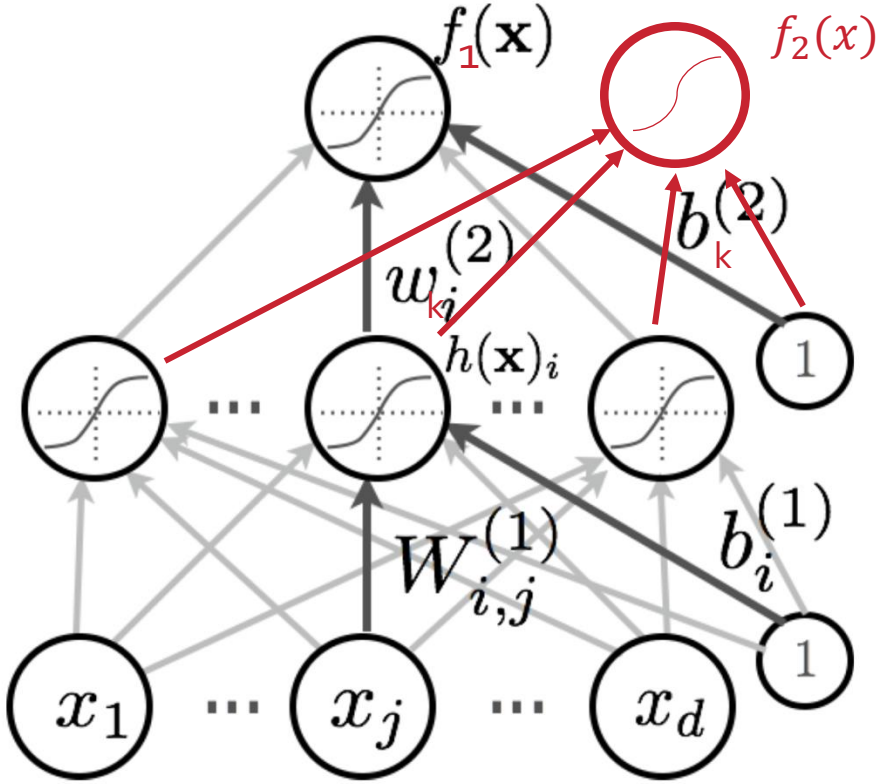
$$f_k(X) = o\left(a^{[2]}\right)_k$$

with the activations $a_k^{[2]} = b_k^{[2]} + \sum_i^H w_{ki}^{[2]} h_i(X)$.

- A classifier can be defined as $G(x) = \mathrm{argmax}_k f_k(x)$.
- Ground truth values are one-hot encoded (a vector of all 0 and 1 for the correct class).

Single output:

$$f(X) = o\left(a^{[2]}\right)$$
$$= o\left(b^{[2]} + \sum_i^H w_i^{[2]} h_i(X)\right)$$



Hertie School

Output layer activation function

Multiclass classification

Our network has the form

$$f_k(X) = o\left(a^{[2]}\right)_k$$

with $a_k^{[2]} = b_k^{[2]} + \sum_i^H w_{ki}^{[2]} h_i(X)$.

For K-fold classification, we use the **softmax function** o: $\mathbb{R}^K \to (0,1)^K$ as an output activation function

$$o\left(a^{[2]}\right)_k = \frac{e^{a_k^{[2]}}}{\sum_{l=1}^K e^{a_l^{[2]}}}.$$

scales output so
that vector values
sum to 1
fulfills the axioms of probability
becomes %s
much more intuitive
we could use sigmoid activation function, but then they wouldn't sum to 1

Example:

$$\begin{bmatrix} f_{dog} \\ f_{cat} \end{bmatrix} = o\left(a^{[2]}\right) = o\left(\begin{bmatrix} 1.2 \\ 0.3 \end{bmatrix}\right) = \begin{bmatrix} \dfrac{\exp 1.2}{\exp(1.2) + \exp(0.3)} \\ \dfrac{\exp 0.3}{\exp(1.2) + \exp(0.3)} \end{bmatrix} = \begin{bmatrix} 0.71 \\ 0.29 \end{bmatrix}$$

- Think of output like probabilities since $\sum_{k=1}^K o\left(a^{[2]}\right)_k = 1$
- We can also use the sigmoid activation function but then the outputs don't sum to 1.

Training neural networks

Data point (x_i)

Model

Neural network

Predicted label ($\hat{y}_i$)

43% Dog
57% Cat

Loss ($\hat{y}_i, y_i$)

Updating weights through back-propagation and gradient descent

Which network fits our data best?

Loss function depends on task we are looking at / what we want to optimise

Loss function for regression

Sum of **squared errors**:

$$L(\theta) = L(f(X), y) = \sum_{i=1}^{N} \left(y_i - f(x_i)\right)^2$$

The sum of squared errors (or mean square error) is often used.

There are also other loss functions that are less commonly used such as:

- Mean absolute error optimises in a more robust way - more robust to outliers
- Mean absolute percentage error
- Mean squared logarithmic error
- Cosine similarity
- …

Loss function

Loss function for (K-class) classification

Sum of **squared errors**:

$$L(\theta) = L(f(X), y) = \sum_{i=1}^{N} \sum_{k=1}^{K} \left(y_{ik} - f_k(x_i)\right)^2$$

could use sum of squared errors in theory,
but it's not ideal

Cross-entropy loss: more often used

$$L(\theta) = -\sum_{i=1}^{N} \sum_{k=1}^{K} y_{ik} \log f_k(x_i)$$

we have ground truth values, and our predicted values
then we multiply them and take the log
and then ????????

- Measures how different the estimated distribution is from the distribution of ground truth values.
- The ground-truth values are one-hot encoded.
- Cross-entropy loss is the most commonly used loss for classification

measures how different the ground truth values are to the estimated values
this is most commonly used for classification

Hertie School

Capacity of a neural network

= it's expressiveness

Salakhutdinov's slides

Artificial neuron

Capacity of a single neuron

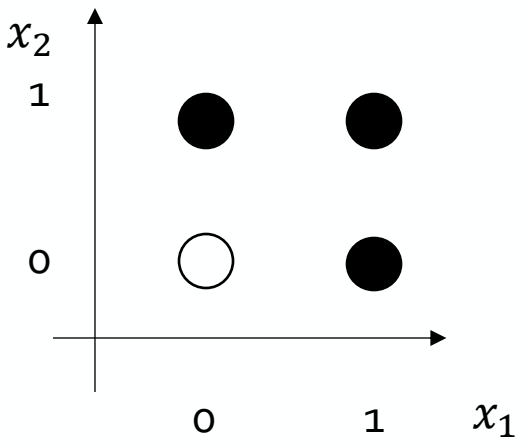
If we apply the following activation function to our neuron:

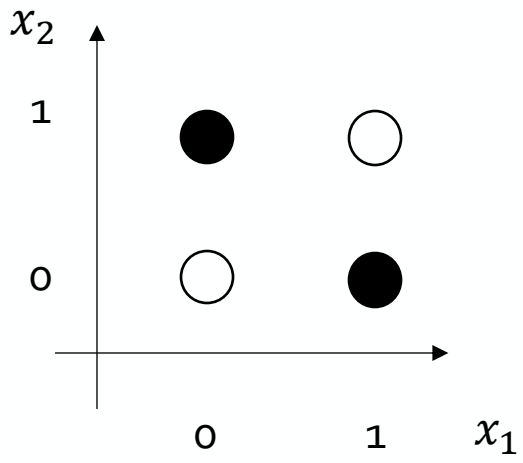
$$g\big(a(X)\big) = \frac{1}{1 + e^{-a(X)}},$$

and recall

$$h(X) = g\big(a(X)\big) = g\left(b + \sum_{j}^{d} w_j x_j\right).$$

Which of the following problems can it learn to solve? (Image: Filled dots mean $y = 1$, empty mean $y = 0$.)





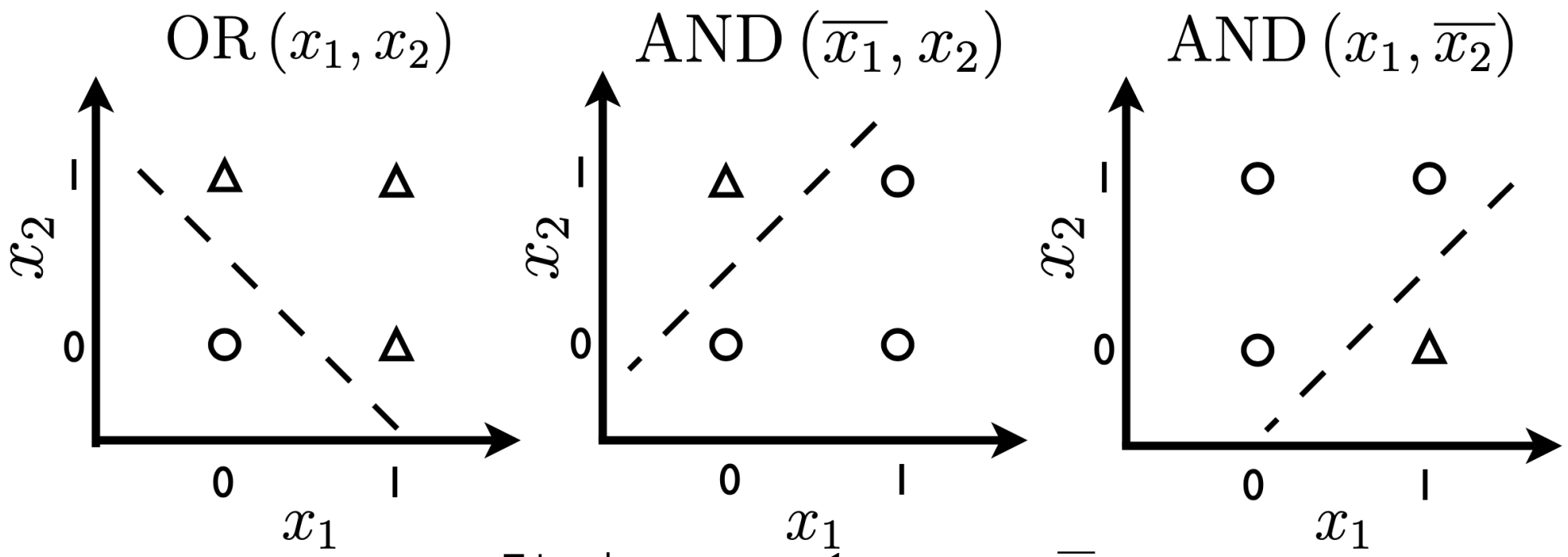
we cannot learn the lower problem because we cannot draw a linear decision boundary to separate these classes - so it cannot be learnt by a single neuron; we'd need more neurons to learn this

Capacity of a single neuron

Linearly separable problems

Can solve linearly separable problems

Neuron with sigmoid activation can be interpreted as estimating $P(y = 1|X)$ (logistic classifier)



Triangles mean $y = 1$,
circles mean $y = 0$

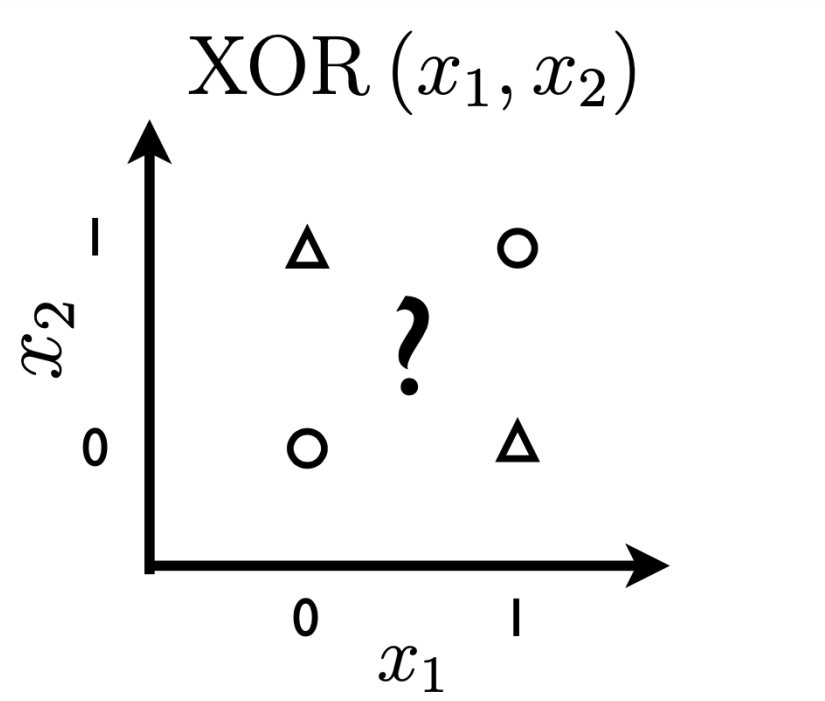
$\overline{x_1}$ means
"not $x_1 = 1$"

Image: Taken from Salakhutdinov

Capacity of a single neuron

Non-linearly separable problems

Cannot solve non-linearly separable problems.



Left: Original "either or" problem
Right: transformation into a linearly separable problem

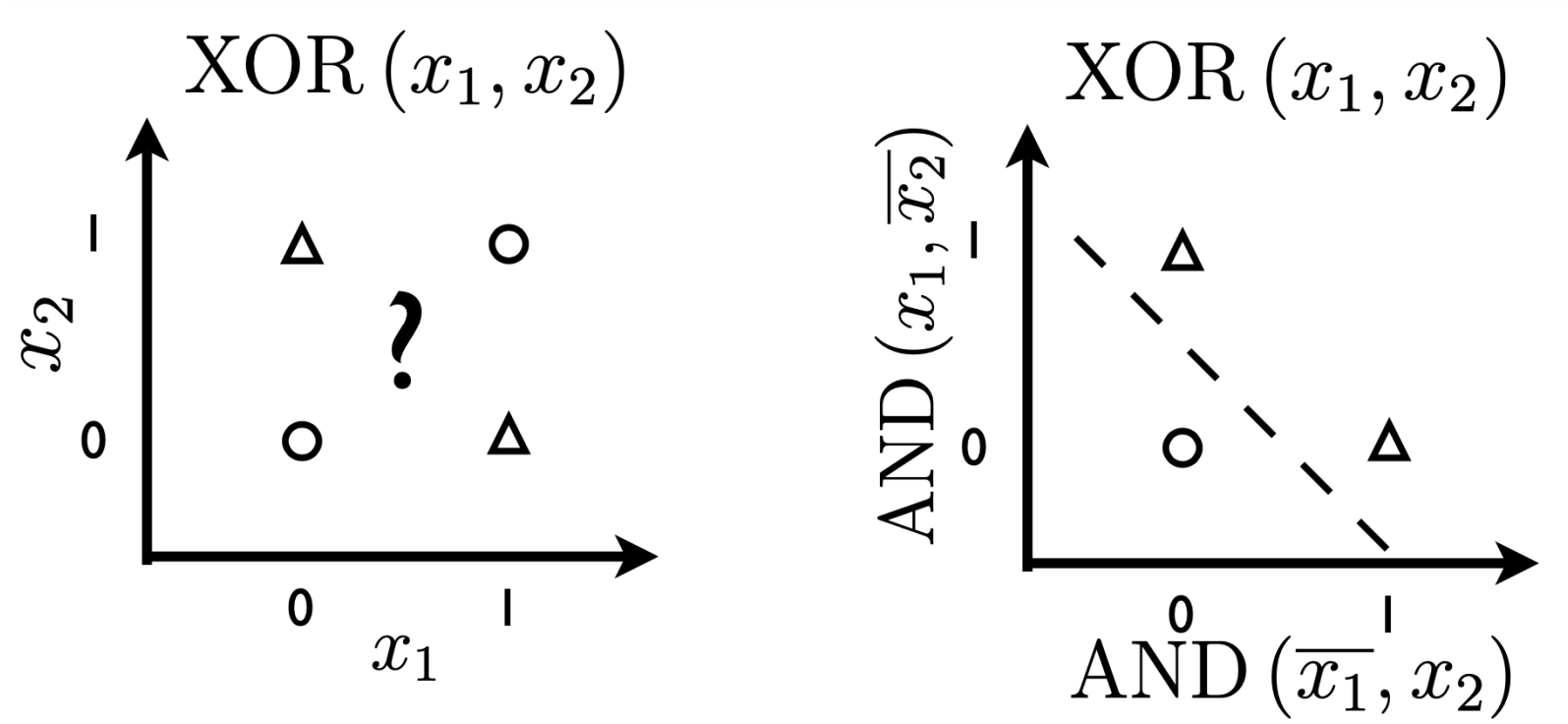
we have to transform this to make it linearly separable

Image: Taken from Salakhutdinov

Capacity of a single neuron

Non-linearly separable problems

Cannot solve non-linearly separable problems.



Left: Original "either or" problem
Right: transformation into a linearly separable problem

we transform the input features
to make it linearly separable
i'm not sure what the transformation is thoug

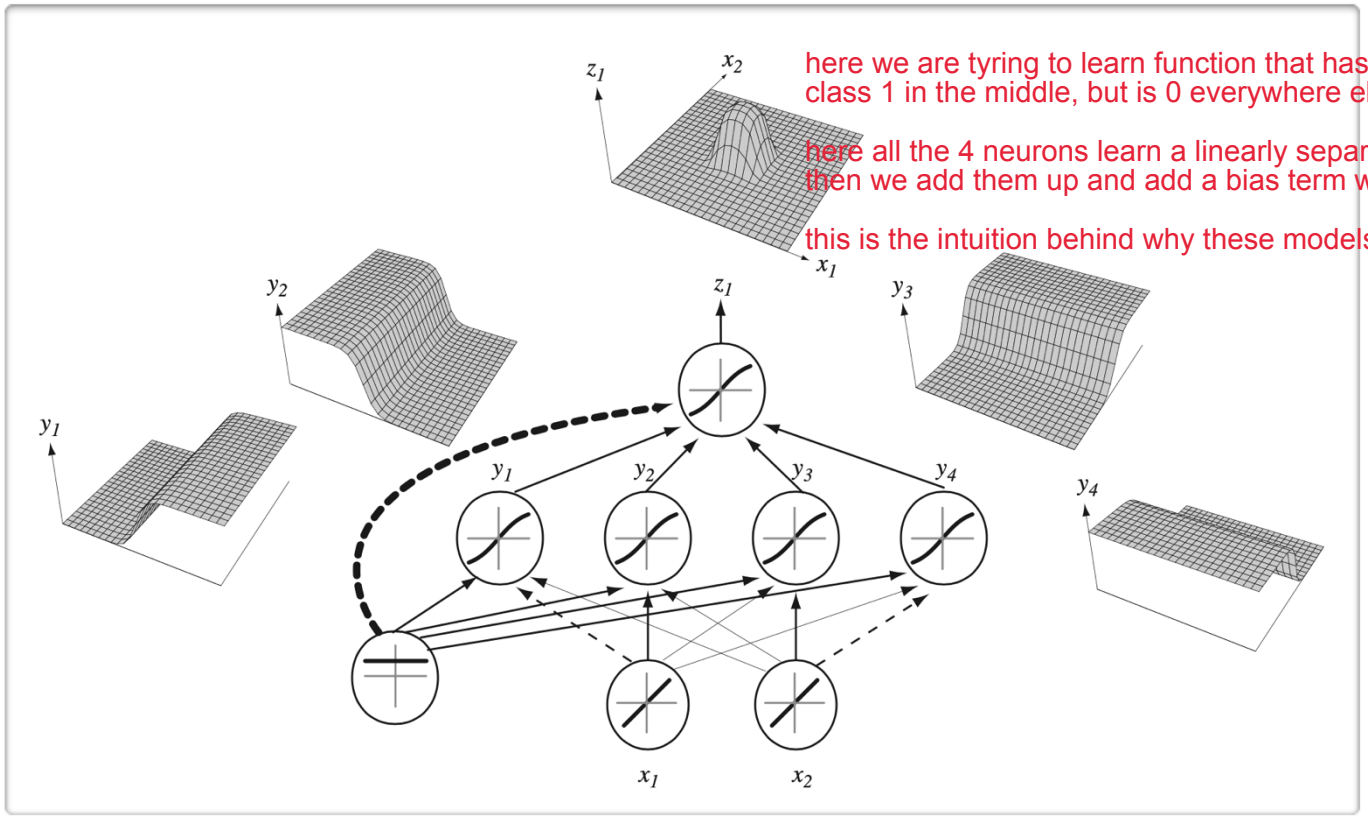
Image: Taken from Salakhutdinov

Hertie School

Capacity of a single-layer neural net

Single-layer neural network

Illustration of how a single layer neural network can represent a function with activations:



here we are tyring to learn function that has class 1 in the middle, but is 0 everywhere else

here all the 4 neurons learn a linearly separable function
then we add them up and add a bias term we get this linearly separable surface

this is the intuition behind why these models work so well

Image: Taken from Salakhutdinov

(from Pascal Vincent's slides)

Universal approximator

Single-layer neural network

Universal Approximation Theorem (Hornik, 1991):

"a single hidden layer neural network with a linear output unit can approximate any continuous function arbitrarily well, given enough hidden units"

- This applies for sigmoid, tanh and many other activation functions.
- However, this does not mean that there is learning algorithm that can find the necessary parameter values.

Hertie School

Gradient descent

Introduction

Why do we need gradient descent?

- In statistical learning we want to find the function (model) $f(X)$ that **minimizes the prediction loss** (expressed by using one of various loss functions $L(f(X,\theta),Y)$)
- Statistical models are a function of the data and **many parameters** $f(X,\theta)$
- In deep learning, neural networks easily have billions of parameters
- Following the calculus approach of setting partial derivatives to zero, we would end up with as many equations $\frac{\partial l}{\partial \theta_i} = 0$ as there are parameters (computationally intractable).
- **Gradient descent**: Approaching the minimum step by step along the direction of steepest descent. this is computationally doable
- We only need to compute the gradient, but not set to zero and solve for the parameters. we still need to take the gradiaent wrt to all our parameters.... but we don't need to solve for it (computationally intractable)

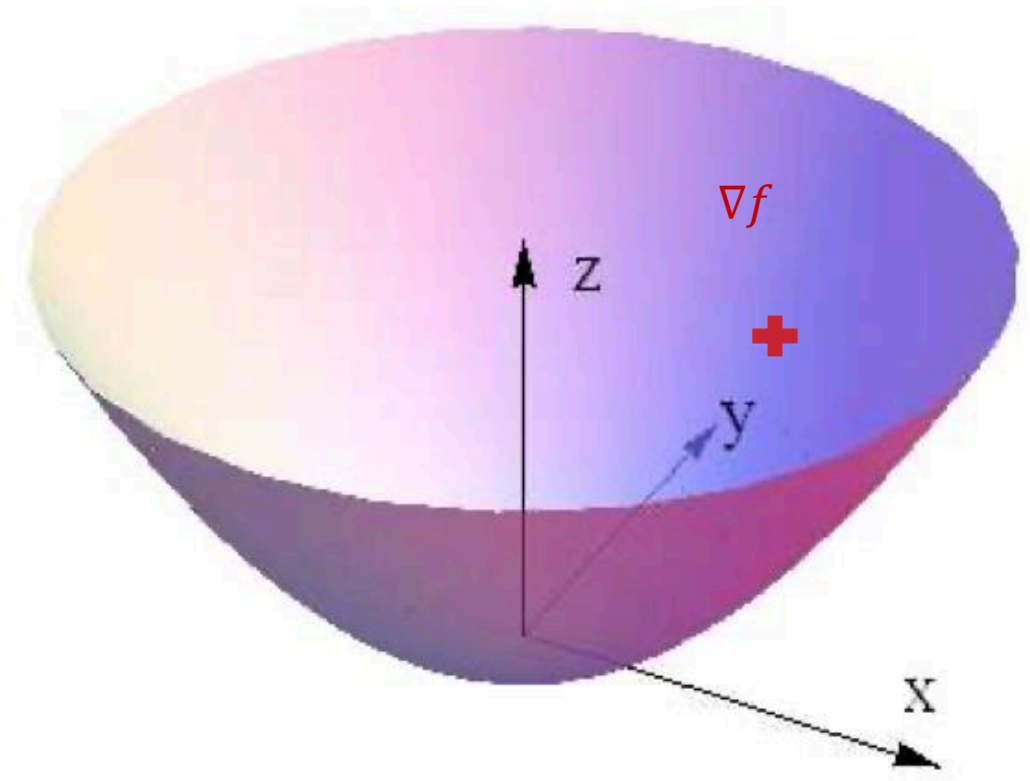
Hertie School

Gradient descent recap

Simple example

Example of a function $f(x, y) = \frac{1}{2}(x^2 + y^2)$.

In which direction does the gradient $\nabla f(x, y)$ point in the red point in the figure on the right?



get rest here from slides

the gradients is just a vector of the partial derivatives
we need to take the negative of this in order to approach the miniumum

Image: Northeastern University

Hertie School

Gradient descent recap

In deep learning we minimize the loss.

Gradient descent method

Given an initial starting point $x^{(0)}$, we want to find a (local) minimum of $f(x)$.

Do until stopping criterion is fulfilled for each step k:

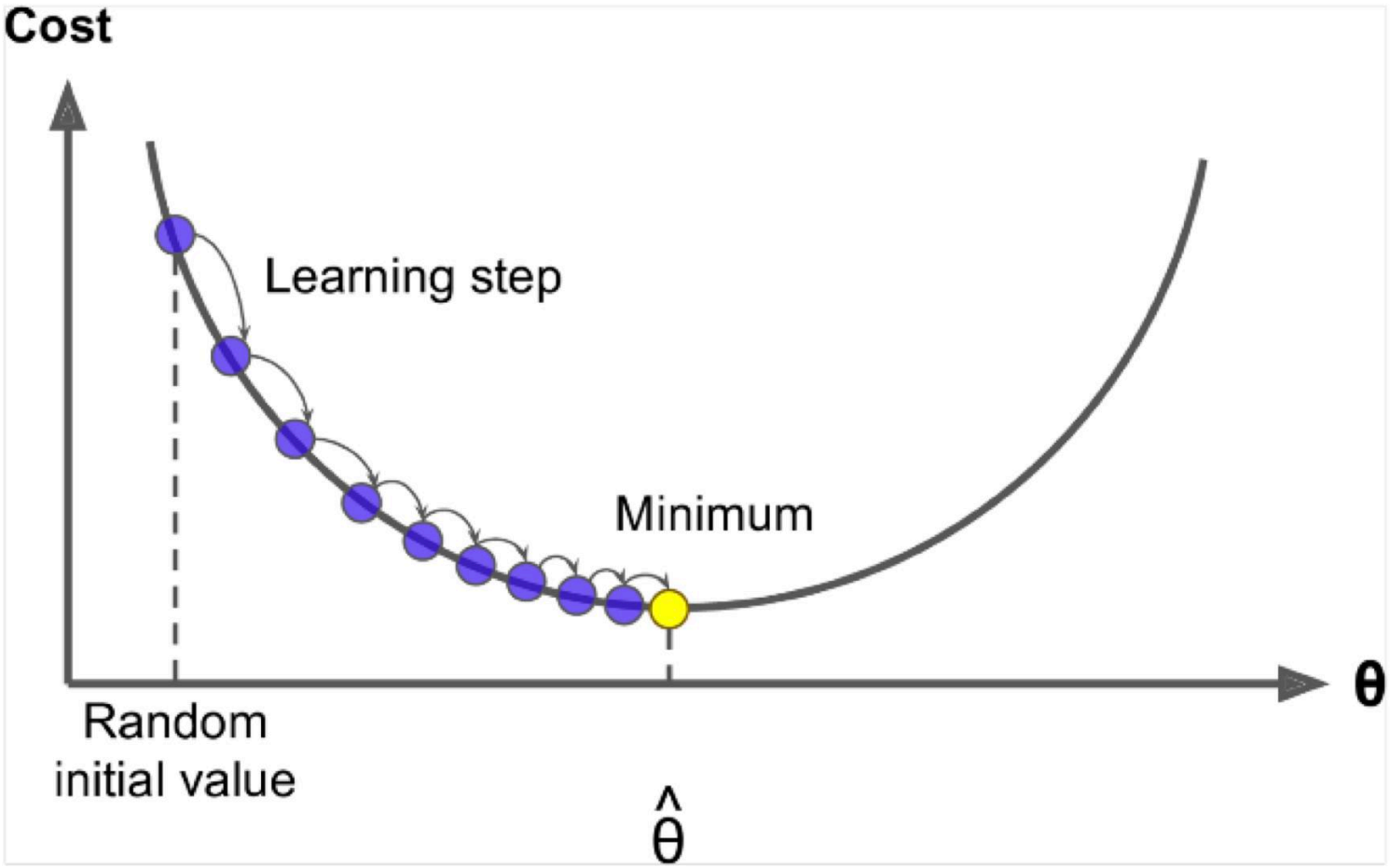
1. Find the gradient (or search direction) $\Delta x^{(k)} = -\nabla f(x^{(k)})$
2. Choose a stepsize $\eta^{(k)}$ (or learning rate)
3. Update with $x^{(k+1)} = x^{(k)} + \eta\, \Delta x^{(k)}$.

Note: x is the vector of variables, $\eta > 0$ is a scalar.

- In gradient descent, we have that $f\left(x^{(k+1)}\right) < f\left(x^{(k)}\right)$ (for some η), exept when $x^{(k)}$ is optimal.
- Possible stopping criterion: $\left\|\nabla f\left(x^{(k)}\right)\right\|_2 \leq \epsilon$, where ϵ is small and positive (check after step 1). in DL we tend to use 'early stopping' instead, where we use the validation data to determine when to stop - to avoid overfitting we usually stop way before we reach the minimum training error... for generalizability

Gradient descent examples

Univariate example from ML class



only with convex functions would we be guaranteed to reach the global minumum using gradient descent

Hertie School

Gradient descent examples

Bivariate example

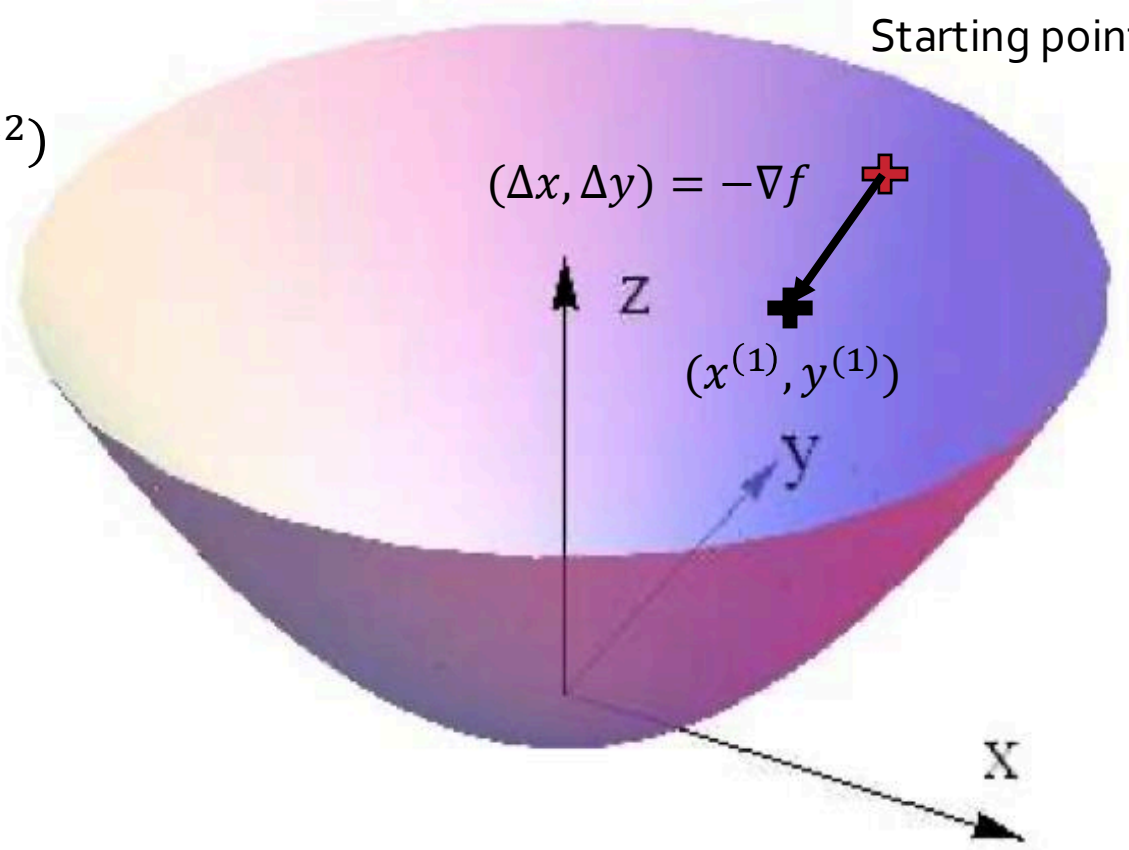
Step 1

$$f(x,y) = \frac{1}{2}(x^2 + y^2)$$

Do until stopping criterion is fulfilled for each step k:

1. Find the gradient (or search direction) $\Delta x^{(k)} = -\nabla f(x^{(k)})$
2. Choose a stepsize $\eta^{(k)}$ (or learning rate)
3. Update with $x^{(k+1)} = x^{(k)} + \eta \Delta x^{(k)}$.

Assume fixed η.



$$\begin{bmatrix} x \\ y \end{bmatrix}^{(k+1)} = \begin{bmatrix} x \\ y \end{bmatrix}^{(k)} - \eta \begin{bmatrix} x \\ y \end{bmatrix}^{(k)}$$

Image: Northeastern University

Gradient descent examples

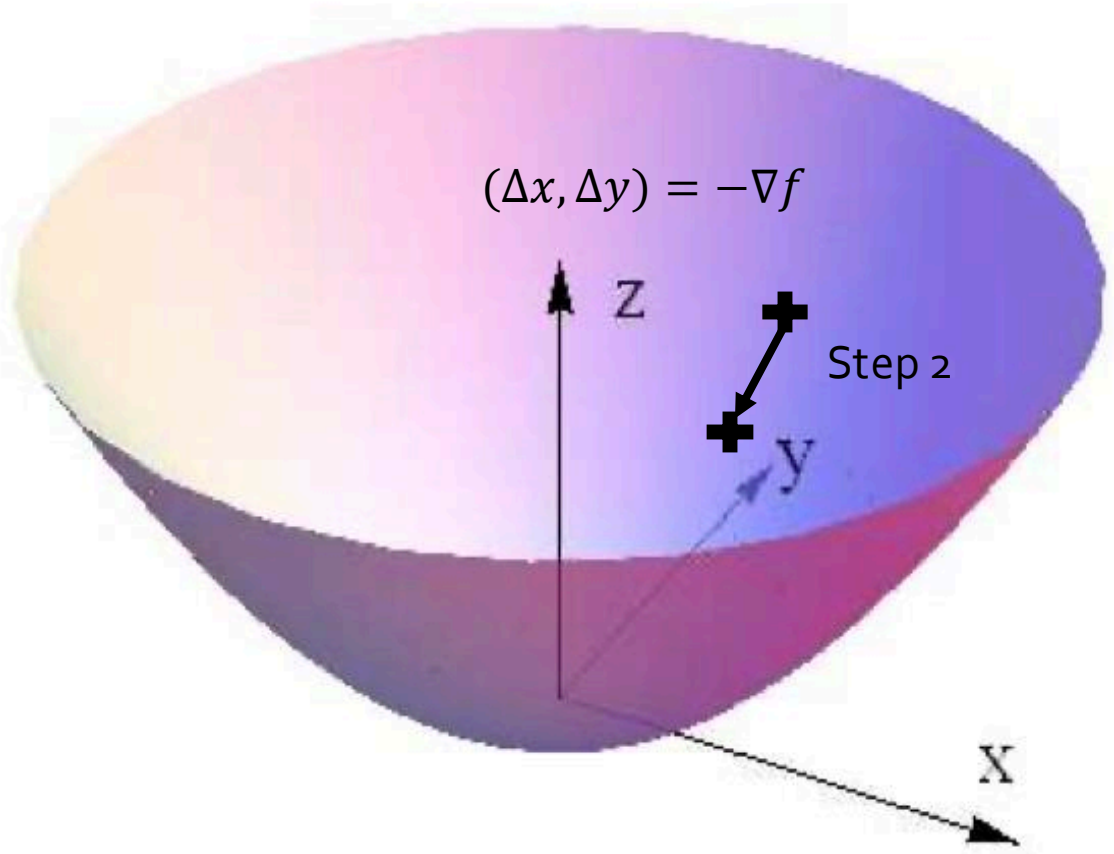
Bivariate example

Step 2

Do until stopping criterion is fulfilled for each step k:

1. Find the gradient (or search direction) $\Delta x^{(k)} = -\nabla f(x^{(k)})$
2. Choose a stepsize $\eta^{(k)}$ (or learning rate)
3. Update with $x^{(k+1)} = x^{(k)} + \eta \Delta x^{(k)}$.

Assume fixed η.



$$\begin{bmatrix} x \\ y \end{bmatrix}^{(k+1)} = \begin{bmatrix} x \\ y \end{bmatrix}^{(k)} - \eta \begin{bmatrix} x \\ y \end{bmatrix}^{(k)}$$

Image: Northeastern University

Hertie School

Gradient descent examples

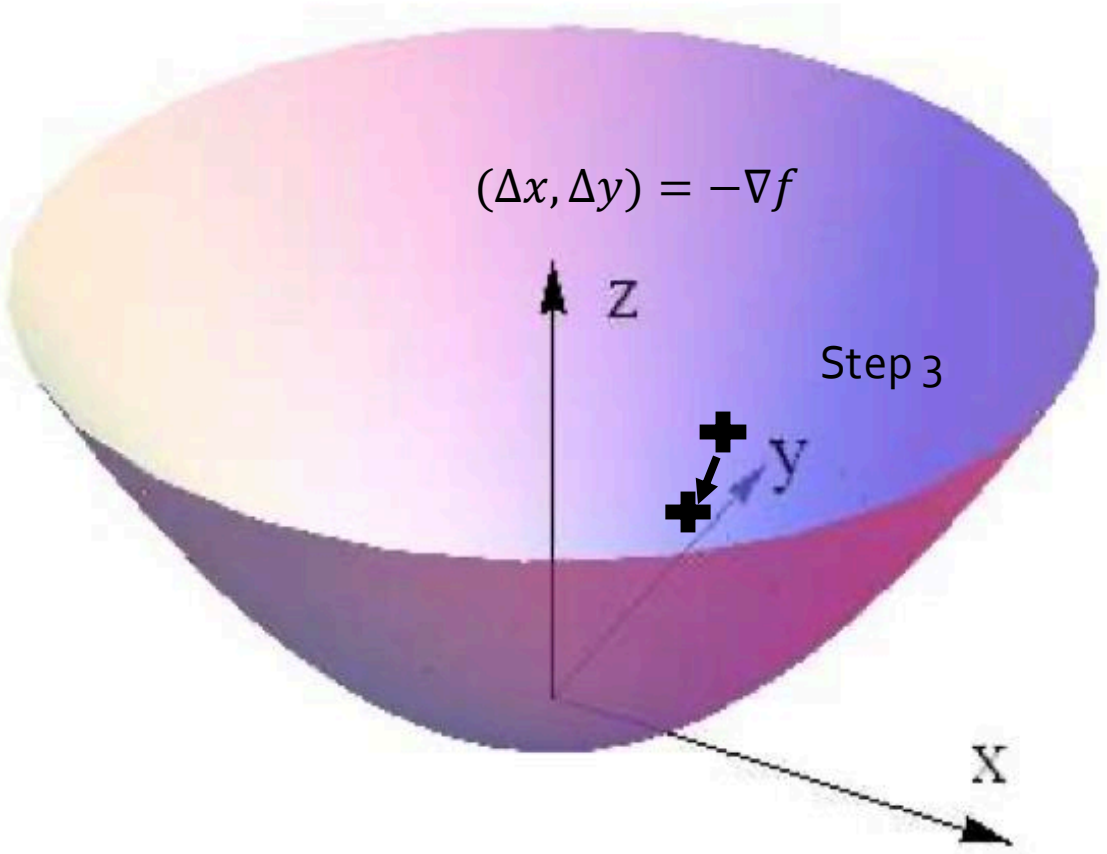
Bivariate example

Step 3

Do until stopping criterion is fulfilled for each step k:

1. Find the gradient (or search direction) $\Delta x^{(k)} = -\nabla f(x^{(k)})$
2. Choose a stepsize $\eta^{(k)}$ (or learning rate)
3. Update with $x^{(k+1)} = x^{(k)} + \eta \Delta x^{(k)}$.

Assume fixed η.



$$\begin{bmatrix} x \\ y \end{bmatrix}^{(k+1)} = \begin{bmatrix} x \\ y \end{bmatrix}^{(k)} - \eta \begin{bmatrix} x \\ y \end{bmatrix}^{(k)}$$

the steps becomes smaller and smaller, because as we are getting closer to 0 the functional values that we plug in will becomes smaller
the gradient evaluated at each successive point becomes smaller
the steps become smaller as the gradients' curve approaches 0, (algebraically the gradient component of the formula

Gradient descent examples

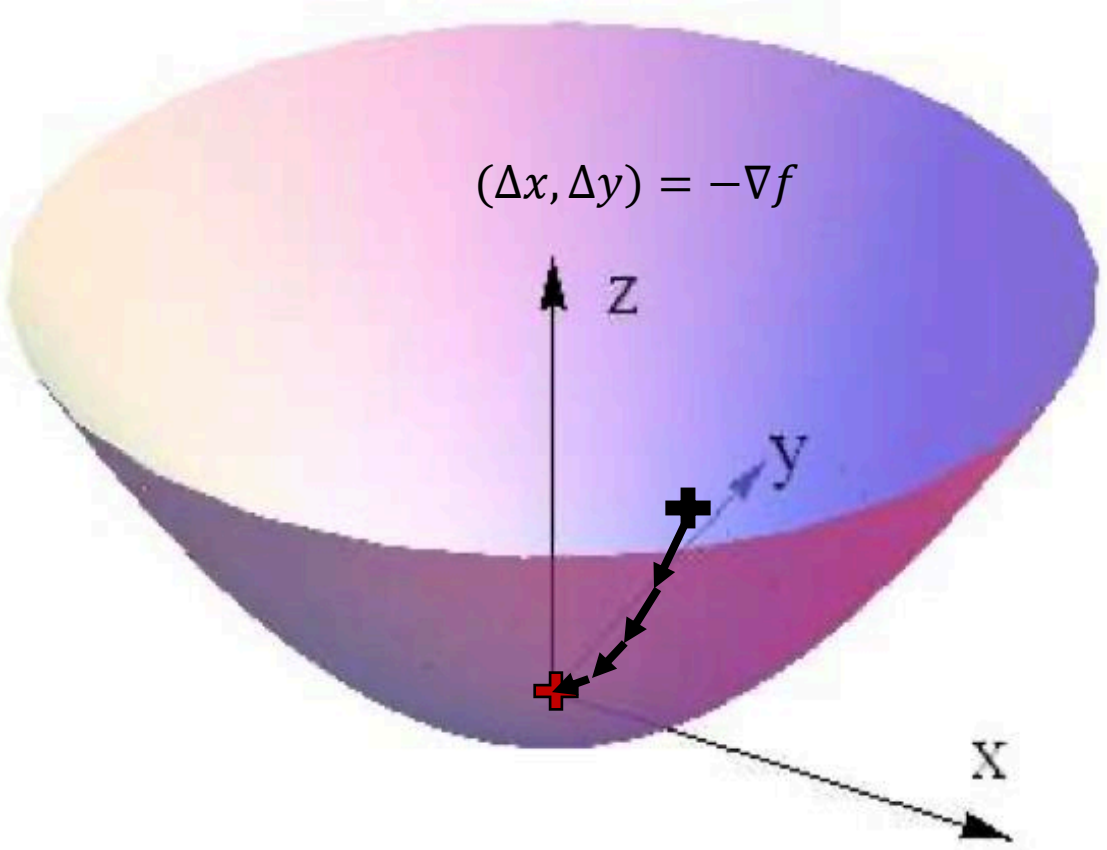
Bivariate example

Converging at the minimum

Do until stopping criterion is fulfilled for each step k:

1. Find the gradient (or search direction) $\Delta x^{(k)} = -\nabla f(x^{(k)})$
2. Choose a stepsize $\eta^{(k)}$ (or learning rate)
3. Update with $x^{(k+1)} = x^{(k)} + \eta \Delta x^{(k)}$.

Assume fixed η.



$$\begin{bmatrix} x \\ y \end{bmatrix}^{(k+1)} = \begin{bmatrix} x \\ y \end{bmatrix}^{(k)} - \eta \begin{bmatrix} x \\ y \end{bmatrix}^{(k)}$$

Image: Northeastern University

Hertie School

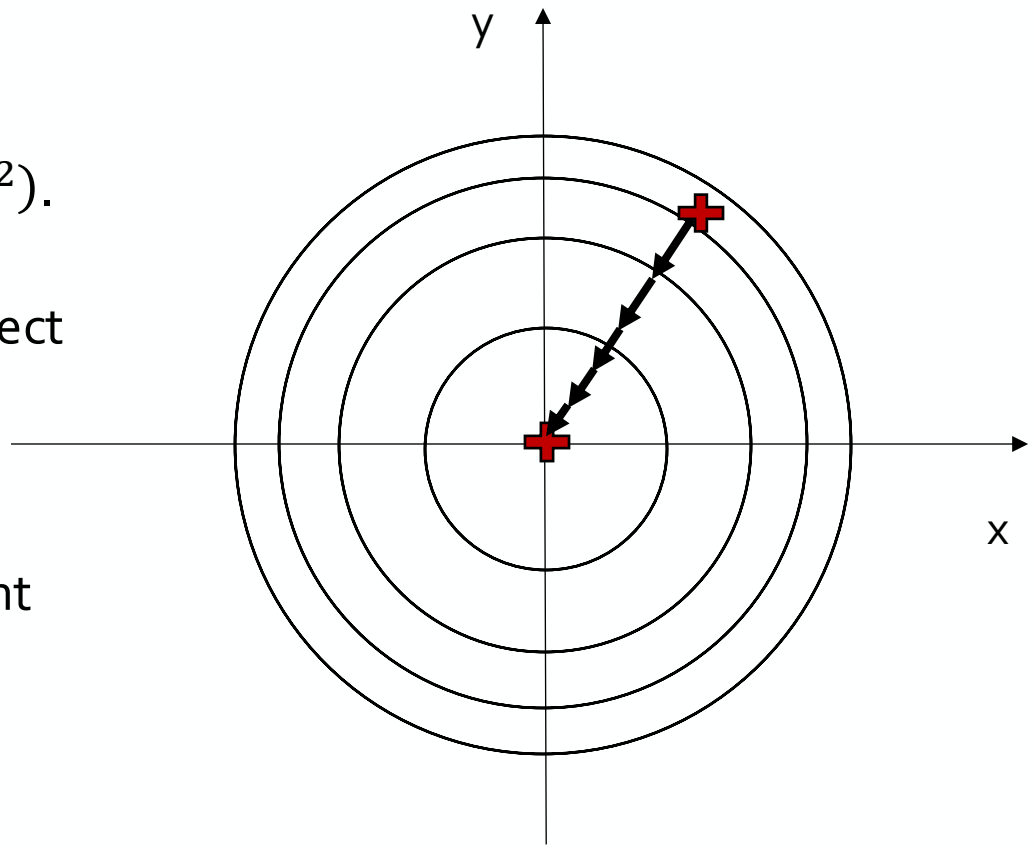
Gradient descent examples

Bivariate example

Contour lines (level set lines) for $f(x, y) = \frac{1}{2}(x^2 + y^2)$.

We see that with gradient descent, we move on a direct path to the minimum.

Note: This is not always the case and gradient descent can be inefficient, or get stuck in local minima.



$$\begin{bmatrix} x \\ y \end{bmatrix}^{(k+1)} = \begin{bmatrix} x \\ y \end{bmatrix}^{(k)} - \eta \begin{bmatrix} x \\ y \end{bmatrix}^{(k)}$$

Hertie School

Training a neural net and backpropagation

Training neural networks

Hertie School

Data point (x_i)

Model

Neural network

Predicted label ($\hat{y}_i$)

43% Dog
57% Cat

Loss ($\hat{y}_i$, y_i)

Updating weights through back-propagation and gradient descent

2) Computing loss and back-propagation

Hertie School

Gradient descent

Gradient descent in neural networks

Given an initial starting point for weights and biases $(W^{(0)}, b^{(0)})$, we want to find a (local) minimum of the loss $L(f(X), y)$.

Example for a weight parameter w. Do until stopping criterion is fulfilled for each step r:

1. Find the gradient (or search direction) $\Delta w^{(r)} = -\nabla L(w^{(r)})$
   1 weight value: a scalar
2. Choose a stepsize $\eta^{(r)}$ (or learning rate)
3. Update with $w^{(r+1)} = w^{(r)} + \eta\, \Delta w^{(r)}$.

we do this at each data point

How do we compute the gradient update $\Delta w^{(r)}$ for a neural network?

Backpropagation!

Hertie School

Training a neural net

Backpropagation overview

Given data X, ground truth y, and model $f(X)$, we estimate the parameters θ (weights and biases) that minimize the loss with **backpropagation**:

1. Compute the functional form of the gradient of the loss function with respect to all parameters θ ← **Detailed in this lecture**
2. Perform stochastic gradient descent to find $\hat{\theta}$ that minimize the loss function by iteratively
    1. doing a **forward-pass** to get the predictions based on the current values of the parameters ← Applying the model to the training data get a prediction

       plug in the data point, see how wrong we are by calculcating the loss
    2. computing the new loss ← Applying the loss function
    3. computing the gradient for current values of the weights and data points/mini-batches in a **backward pass** ← Plugging in the parameter values
    4. updating the parameters ← Update rule from gradient descent using a learning rate

Hertie School

Computing the gradient

The gradient of the loss

Recall the generic form of a neural network with one output (e.g. regression):

$$f(X) = o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} h_i(X)\right) = o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} g\left(b_i^{[1]} + \sum_{j}^{d} w_{ij}^{[1]} x_j\right)\right)$$

Finding an algebraic expresssion of the gradient of the loss for the gradient update:

$$\nabla L\left(X, y; b_i^{[1]}, w_{ij}^{[1]}, b^{[2]}, w_i^{[2]}\right) = \begin{pmatrix} \frac{\partial L}{\partial b_i^{[1]}} \\ \frac{\partial L}{\partial w_{ij}^{[1]}} \\ \frac{\partial L}{\partial b^{[2]}} \\ \frac{\partial L}{\partial w_i^{[2]}} \end{pmatrix}$$

(simplified)

Key ingredients:

- Chain rule
- Derivatives of loss functions
- Derivatives of activation functions
- Derivatives of output activations

Hertie School

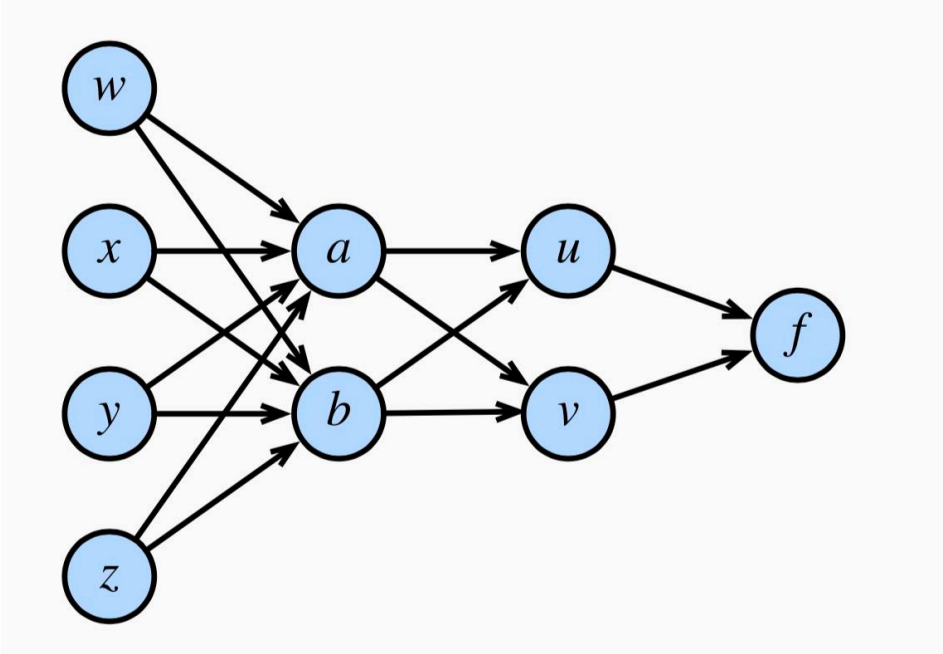
Computing the gradient

Multivariate chain rule

- Chains of multivariate functions are common in neural networks
- Chain rule is key ingredient for backpropagation
- Graphical representation: Nodes=variables, edges=functional dependence
- $f(u, v)$ but also $f(a, b)$ and $f(w, x, y, z)$
- **Multivariate chain rule**:

$$\frac{\partial}{\partial a} f(u(a,b), v(a,b)) = \frac{\partial f}{\partial u}\frac{\partial u}{\partial a} + \frac{\partial f}{\partial v}\frac{\partial v}{\partial a}$$

we have to add all the paths between a and f



- Typical for backpropagation $\frac{\partial f}{\partial w}$?

Image source: Zhang et al.

Solution: $\frac{\partial f}{\partial w} = \frac{\partial f}{\partial u}\frac{\partial u}{\partial a}\frac{\partial a}{\partial w} + \frac{\partial f}{\partial v}\frac{\partial v}{\partial a}\frac{\partial a}{\partial w} + \frac{\partial f}{\partial u}\frac{\partial u}{\partial b}\frac{\partial b}{\partial w} + \frac{\partial f}{\partial v}\frac{\partial v}{\partial b}\frac{\partial b}{\partial w}$

in multivariate setting, we have to consider all the variables

Computing the gradient

Hertie School

Single-layer neural network simplified unraveled

$$f(X) = o\left(a^{[2]}\right)$$

this is the last layer which contains all the layers output * final activation

$$= o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} h_i^{[1]}\right)$$

here we also have all the hidden units h

$$= o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} g\left(a_i^{[1]}\right)\right)$$

here we know what goes into h, it's theactivations from the prev year

$$= o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} g\left(b_i^{[1]} + \sum_{j}^{d} w_{ij}^{[1]} x_j\right)\right)$$

Computing the gradient update for weight $b_i^{[1]}$ with linear output $o()$

$$\frac{\partial L(f(X), y)}{\partial b_i^{[1]}}$$

$$= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial h_i^{[1]}} \cdot \frac{\partial h_i^{[1]}}{\partial a_i^{[1]}} \cdot \frac{\partial a_i^{[1]}}{\partial b_i^{[1]}}$$

wrt 1st activation

then we realise that the activation fn is a function of the h

here we can compute the derivatives of each one of these rather than thinking about it all at once

Hertie School

Computing the gradient

Weight update example

$$f(X) = o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} g\left(b_i^{[1]} + \sum_{j}^{d} w_{ij}^{[1]} x_j\right)\right)$$

Computing the gradient update for a weight w_{ij} using the chain rule (with linear output $o()$):

$$\frac{\partial L(f(X), y)}{\partial w_{ij}} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial h_i^{[1]}} \cdot \frac{\partial h_i^{[1]}}{\partial a_i^{[1]}} \cdot \frac{\partial a_i^{[1]}}{\partial w_{ij}^{[1]}}$$

Now we can compute the gradient more easily by computing each factor separately. Many of the partial derivatives are known.

Hertie School

Computing the gradient

Derivatives of common activation functions

Sigmoid

$$g(z) = \sigma(z) = \frac{e^z}{1 + e^z}$$

$$g'(z) = \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Tanh

$$g(z) = \tanh z = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - \tanh^2(z)$$

ReLu

$$g(z) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{otherwise} \end{cases}$$

Hertie School

Computing the gradient

Derivative of the softmax

Softmax output layer in single-layer neural network for K-class classification

$$o(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

Derivative

$$\frac{\partial o(z)_i}{\partial z_j} = \begin{cases} o(z)_i\left(1 - o(z)_j\right) & i = j \\ -o(z)_i o(z)_j & i \neq j \end{cases}$$

Or using the Kronecker delta

$$\frac{\partial o(z)_i}{\partial z_j} = o(z)_i(\delta_{ij} - o(z)_j)$$

Kronecker delta:

$$\delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

Hertie School

Example

Single-layer neural net with squared error loss, linear output fn, and sigmoid activation fn

The network:

$$f(X) = b^{[2]} + \sum_{i}^{H} w_i^{[2]} \sigma\left(b_i^{[1]} + \sum_{j}^{d} w_{ij}^{[1]} x_j \right)$$

The loss for one data point (x,y):

$$L(f(X), y) = \left(y - f(x)\right)^2$$

The gradient for weight w_{ij}:

$$\frac{\partial L(f(X), y)}{\partial w_{ij}} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial h_i^{[1]}} \cdot \frac{\partial h_i^{[1]}}{\partial a_i^{[1]}} \cdot \frac{\partial a_i^{[1]}}{\partial w_{ij}^{[1]}}$$

Example

$$\frac{\partial L(f(X),y)}{\partial w_{ij}} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial h_i^{[1]}} \cdot \frac{\partial h_i^{[1]}}{\partial a_i^{[1]}} \cdot \frac{\partial a_i^{[1]}}{\partial w_{ij}^{[1]}}$$

Term 1 (derivative of loss):

$$\frac{\partial L}{\partial f} = \frac{\partial}{\partial f}\left(y - f(x)\right)^2 = -2(y - f)$$

Term 2 (derivative of linear output):

$$\frac{\partial f}{\partial a^{[2]}} = \frac{\partial a^{[2]}}{\partial a^{[2]}} = 1$$

Term 3 (derivative of output activation):

$$\frac{\partial a^{[2]}}{\partial h_i^{[1]}} = \frac{\partial}{\partial h_i^{[1]}}\left[b^{[2]} + \sum_l^H w_l^{[2]} h_l^{[1]}\right] = w_i^{[2]}$$

Term 4 (derivative of activation function):

$$\frac{\partial h_i^{[1]}}{\partial a_i^{[1]}} = \frac{\partial \sigma\left(a_i^{[1]}\right)}{\partial a_i^{[1]}} = \sigma\left(a_i^{[1]}\right)\left(1 - \sigma\left(a_i^{[1]}\right)\right)$$

Term 5 (derivative for hidden layer activation):

$$\frac{\partial a^{[1]}}{\partial w_{ij}^{[1]}} = \frac{\partial}{\partial w_{ij}^{[1]}}\left[b_i^{[1]} + \sum_m^d w_{im}^{[1]} x_m\right] = x_j$$

Example

Putting it all together

$$\frac{\partial L(f(X),y)}{\partial w_{ij}} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial h_i^{[1]}} \cdot \frac{\partial h_i^{[1]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial w_{ij}^{[1]}}$$

$$= -2(y - f(x))w_i^{[2]}\sigma\left(a^{[1]}\right)\left(1 - \sigma\left(a^{[1]}\right)\right)x_j$$

Gradient update:

We plug in all the values for the rth iteration of

- the other weights
- all input features of the data
- the ground truth and the prediction

We update the weight using the gradient and the learning rate η

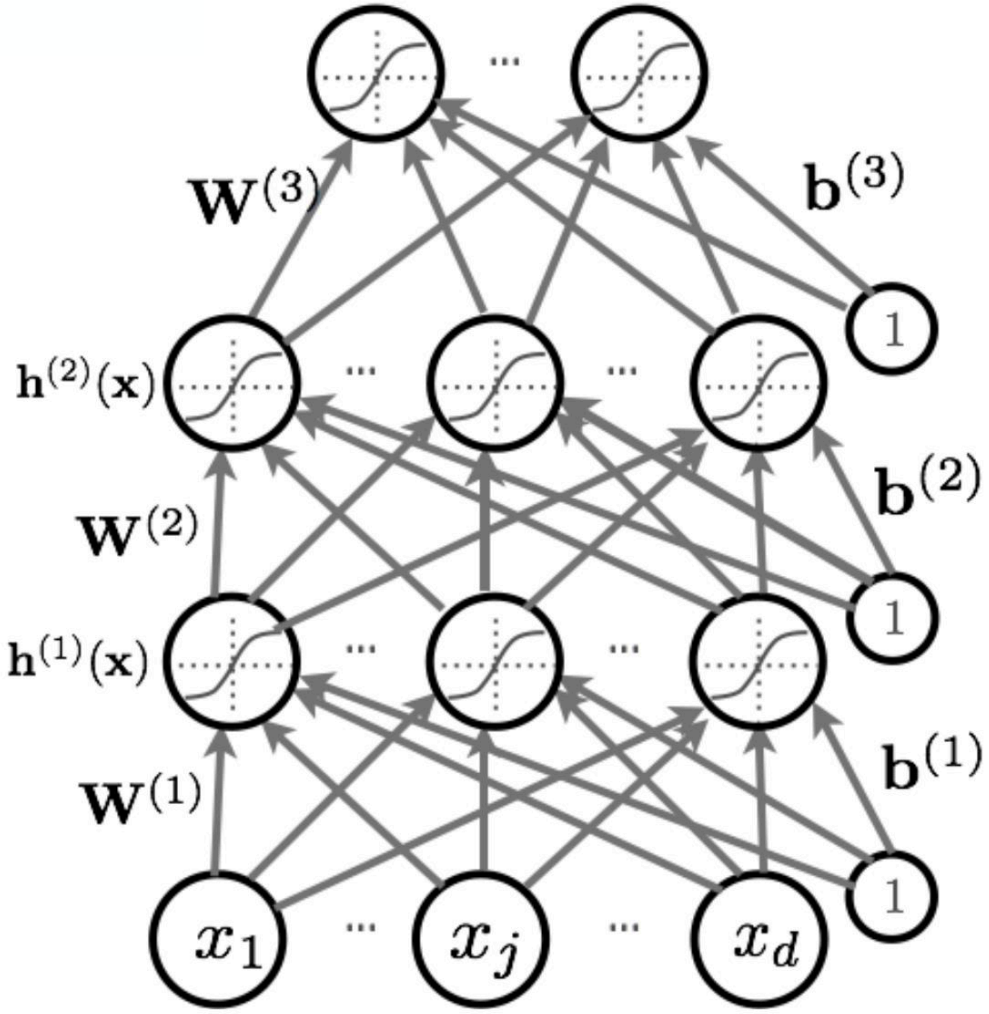
$$w_{ij}^{(r+1)} = w_{ij}^{(r)} + \eta\, \Delta w_{ij}^{(r)} = w_{ij}^{(r)} - \eta \left(\frac{\partial L(f(X),y)}{\partial w_{ij}}\right)^{(r)}$$

Learning outcomes

Hertie School

What we have covered

- Overview of how neural networks "learn"
- Structure of single-layer neural networks
- Gradient descent
- Training the network and back-propagation



Hertie School

Hertie School
Friedrichstraße 180
10117 Berlin, Germany
T +49 (0)30 259219-0
F +49 (0)30 259219-11
info@hertie-school.org
www.hertie-school.org